

УДК Lorem ipsum dolor sit amet

Операционная семантика выражений в языке Go на языке ABML

Харьков А.А. (Студент ММФ НГУ)

Ануреев И.С. (Институт систем информатики СО РАН)

В данной статье представлено формальное описание операционной семантики выражений языка программирования Go с использованием языка спецификаций ABML. Предложенная онтологическая модель позволяет строго задать семантику базовых конструкций языка и может служить основой для разработки инструментов верификации программ.

В тексте статьи для визуального разделения двух используемых языков программирования применяется стиль с цветными боковыми полосами:

- Многострочный код на языке **Go** выделяется *светло-зелёным фоном и тёмно-зелёной боковой полосой*;
- Многострочный код на языке **ABML** выделяется *белым фоном и синей боковой полосой*.

Ключевые слова: операционная семантика, язык Go, язык ABML, формальные методы, онтологическое моделирование.

1. Введение

Формальное описание семантики языков программирования является фундаментальной задачей, лежащей в основе разработки компиляторов, статических анализаторов и инструментов верификации программ. Традиционные подходы к спецификации семантики, основанные на естественном языке, часто страдают от неоднозначности и неполноты, что затрудняет создание формально верифицируемых инструментов.

В данной работе рассматривается фрагмент языка программирования Go и его операционная семантика, формализованная с помощью языка спецификаций ABML (Agent-Based Modeling Language). Выбор языка Go обусловлен его возрастающей популярностью в промышленной разработке, особенно в области системного программирования и создания распределённых приложений. Язык ABML, в свою очередь, предоставляет средства для описания онтологий предметных областей и формализации правил перехода между состояниями.

При построении модели на фрагмент языка Go накладываются следующие ограничения, не снижающие общности подхода, но упрощающие формализацию:

1. Использование обобщённых типов (дженериков) не допускается. Поскольку язык Go является строго типизированным, компилятор выполняет инстанцирование шаблонных типов на этапе компиляции, что позволяет заменить обобщённые конструкции конкретными экземплярами;
2. Имена полей структур должны быть уникальными в пределах одной структуры. Это ограничение является естественным для языка Go и разрешимо на этапе предварительной обработки путём переименования конфликтующих полей.

Основной вклад работы заключается в разработке онтологической модели базовых конструкций языка Go и формальном описании их операционной семантики. Предложенный подход может служить основой для создания инструментов статического анализа, верификации программ и автоматической генерации тестов.

2. Онтология фрагмента языка Go

Построение формальной модели семантики языка программирования требует предварительного создания онтологии — системы понятий, описывающих синтаксические конструкции и семантические сущности языка. В данном разделе на языке ABML вводятся типы объектов, соответствующие элементам фрагмента языка Go: переменным, функциям, операторам, декларациям и т.д.

Классификация и порядок изложения опираются на официальную спецификацию языка Go [?]. Большинство наименований моделируемых объектов соответствуют их названиям в спецификации для удобства восприятия и облегчения поиска дополнительной информации. В отдельных случаях, однако, некоторые сущности были признаны излишними с точки зрения моделирования их функциональности, а отдельные имена заменены на более точно отражающие семантику описываемых объектов.

2.1. Имена

Имена объектов в языке Go являются важнейшей категорией, связывающей синтаксические конструкции с семантическими сущностями. Ниже вводятся типы, моделирующие имена различных объектов программы. Для понимания приводимых определений необходимо знакомство с основными конструкциями языка ABML. Ключевое слово `typedef`

вводит новый тип, конструкция `(uniont ...)` определяет объединение типов, а `mot` (Model of Type) — тип объекта с именованными атрибутами.

```
1 (typedef "identifier" (uniont string))
2 (typedef "variable name" "identifier")
3 (typedef "field name" "identifier")
4 (typedef "function name" "identifier")
5 (typedef "method name" "identifier")
6 (typedef "interface name" "identifier")
7 (typedef "label name" "identifier")
8 (typedef "type name" "identifier")
```

Тип `"identifier"` представляет общее понятие имени объекта в программе. Остальные типы конкретизируют это понятие для различных категорий объектов: `"variable name"` — для переменных, `"field name"` — для полей структур, `"function name"` — для функций, `"method name"` — для методов, `"interface name"` — для интерфейсов, `"label name"` — для меток переходов, `"type name"` — для имён типов.

2.2. Значения и память

Моделирование памяти и значений является основой для описания операционной семантики. В силу строгой типизации языка Go каждый объект программы имеет явно определённый тип, что позволяет однозначно сопоставлять значениям их типы.

2.2.1. Ячейки памяти и значения

```
1 (mot "location" :at "type" "type" :at "value" "Go value")
2 (typedef "Go value" (uniont "literal" "location"))
```

Тип `"location"` описывает ячейку памяти — абстракцию, связывающую тип хранимых значений с их текущим содержимым. Такое представление позволяет моделировать как простые переменные, так и составные объекты, размещённые в памяти. Тип `"Go value"` объединяет все возможные значения языка Go, которые могут быть либо литералами (непосредственно заданными значениями), либо ссылками на ячейки памяти.

2.2.2. Константные значения

Константные значения, включая комплексные числа, вычисляются на этапе компиляции и моделируются следующим образом:

```

1 (typedef "constant" (uniont "true" "false" int real "complex
   constant" string symbol))
2 (mot "complex constant" :at "re" real :at "im" real)

```

2.2.3. Литералы

Литералы представляют собой значения, которые могут быть непосредственно записаны в исходном коде программы. Для составных типов вводятся соответствующие литералы:

```

1 (typedef "literal" (uniont "constant" "composite lit"
   "function lit" "channel lit"))
2 (typedef "composite literal" (uniont "array lit" "slice lit"
   "struct lit" "map lit"))
3 (mot "array lit" :at "type" "array type" :at "value" (listt
   "Go value"))
4 (mot "slice lit" :at "type" "slice type" :at "value" (listt
   "Go value"))
5 (mot "struct lit" :at "type" "struct type" :at "value" (cot
   :amap "field name" "Go value"))
6 (mot "map lit" :at "type" "map type" :at "value" (cot :amap
   "Go value" "Go value"))

```

Конструкция `cot :amap ...` задаёт ассоциативный массив, сопоставляющий ключам (име на полей или значения) соответствующие значения элементов.

2.2.4. Функциональные литералы

Функциональные литералы описываются отдельно, поскольку функции в языке Go являются значениями первого класса:

```

1 (mot "function lit" :at "signature" "signature" :at "body"
   "block")
2 (mot "signature" :at "parameters" (listt "parameter decl"))

```

```

3      :at "result" (uniont (listt "type") (listt "parameter
      decl"))))
4 (typedef "parameter decl" (uniont "single param decl" "multi
      param decl"))
5 (mot "single param decl" :at "name" "parameter name" :at
      "type" "type")
6 (mot "multi param decl" :at "names" (listt "parameter name")
      :at "type" "type")

```

Тип `"function lit"` моделирует анонимные функции. Сигнатура функции `"signature"` содержит список параметров и типы возвращаемых значений. Декларации параметров разделяются на одиночные (`"single param decl"`) и множественные (`"multi param decl"`). Множественные декларации возникают при объявлении нескольких параметров с общим типом, например, `func(a, b, c int)`. В процессе предварительной обработки все множественные декларации преобразуются в последовательности одиночных для унификации последующей обработки.

2.2.5. Каналы

Для моделирования каналов — механизма межгорутинного взаимодействия — вводится тип `"channel lit"`:

```

1 (mot "channel lit" :at "type" "channel type" :at "cap" nat :at
      "buffer" (listt "Go value"))

```

Атрибут `"cap"` задаёт ёмкость буфера канала, а `"buffer"` представляет собой очередь сообщений.

2.3. Типы данных

Система типов языка Go разделяется на базовые (простые) и составные типы.

2.3.1. Базовые типы

Базовые типы включают логический, числовые, символьный и строковый:

```

1 (typedef "type" (uniont "base type" "composite type"))
2 (typedef "base type" (uniont "boolean type" "numeric type"

```

```

    "rune type" "string type"))
3 (typedef "boolean type" (enumt "bool"))
4 (typedef "numeric type" (uniont "int type" "float type"
    "complex type"))
5 (typedef "int type" (uniont "int" "int8" "int16" "int32"
    "int64"
6                                "uint" "uint8" "uint16" "uint32"
                                "uint64" "uintptr" "byte"))
7 (typedef "float type" (enumt "float32" "float64"))
8 (typedef "complex type" (enumt "complex64" "complex128"))
9 (typedef "rune type" (enumt "rune"))
10 (typedef "string type" (enumt "string"))

```

Конструкция `enumt` задаёт перечислимый тип, значениями которого являются указанные константы.

2.3.2. Составные типы

Составные типы объединяют структуры данных, построенные из базовых или других составных типов:

```

1 (typedef "composite type" (uniont "array type" "slice type"
    "struct type"
2                                "pointer type" "function type"
                                "interface type"
3                                "map type" "channel type"))

```

2.3.3. Массивы и срезы

```

1 (mot "array type" :at "len" nat :at "element type" "type")
2 (mot "slice type" :at "element type" "type")

```

Массивы имеют фиксированную длину, известную на этапе компиляции, в то время как срезы представляют собой динамические представления последовательных фрагментов массивов.

2.3.4. Структуры и указатели

Структуры объединяют набор полей с различными типами:

```
1 (mot "struct type" :at "fields" (cot :amap "field name"
    "type"))
2 (mot "pointer type" :at "type" "type")
```

2.3.5. Функциональные типы

```
1 (mot "function type" :at "signature" "type signature")
2 (mot "type signature" :at "types" (listt "type")
3   :at "variadic type" "type" :at "result" (listt "type"))
```

Тип `"type signature"` описывает сигнатуру функции в терминах типов: список типов параметров `"types"`, тип для вариативного параметра `"variadic type"` (если присутствует) и список типов возвращаемых значений `"result"`.

2.3.6. Интерфейсы

Интерфейсы задают множество методов, которые должны быть реализованы:

```
1 (mot "interface type" :at "elements" (cot :amap "method name"
    "signature"))
```

Пример интерфейса на языке Go:

```
1 type A interface {
2     f(n int)      // "method name" = 'f', "signature" = '(n
    int) '
3 }
```

2.3.7. Словари и каналы

Словари (карты) и каналы завершают систему составных типов:

```
1 (mot "map type" :at "key type" "type" :at "element type"
    "type")
2 (mot "channel type" :at "direction" "direction" :at "type"
```

```

    "type")
3 (typedef "direction" (enumt "bidirectional" "send" "receive"))

```

Направление канала `"direction"` может быть двунаправленным (`bidirectional`), только для отправки (`send`) или только для приёма (`receive`).

2.4. Блоки

Блоки определяют области видимости переменных и меток. В языке Go блок представляет собой последовательность операторов, заключённую в фигурные скобки.

```

1 (mot "block"
2   :at "statements" (listt "statement")
3   ; semantic attributes
4   :at "variable location" (cot :amap "variable name"
5     "location")
6   :at "label position" (cot :amap "label" nat)
7 )

```

Атрибут `"statements"` содержит список операторов блока в модельном представлении. Семантические атрибуты `"variable location"` и `"label position"` служат для управления областями видимости. Атрибут `"variable location"` сохраняет для каждой переменной её ячейку памяти до входа в блок, что позволяет корректно обрабатывать затенение (shadowing) имён переменных. Атрибут `"label position"` сопоставляет меткам их позиции в блоке, обеспечивая корректную обработку операторов перехода.

2.5. Декларации

Декларации вводят новые объекты в программу.

2.5.1. Общее понятие декларации

Общее понятие декларации представлено типом `"declaration"`:

```

1 (typedef "declaration" (uniont "type decl" "var decl"
2   "function decl"
3   "method decl" "interface decl"))

```


2.5.2. Декларация типов

Декларация типа связывает имя с определением типа:

```
1 (mot "type decl" :at "name" "type name" :at "type" "type")
```

Пример на языке Go:

```
1 type Point struct{ x bool }
```

2.5.3. Декларация переменных и констант

```
1 (mot "const decl block" :at "declarations" (listt "var decl"))
2 (mot "var decl" :at "names" (listt "variable name")
3   :at "types" (listt "type") :at "values" (listt
   "expression"))
```

Тип `"const decl block"` моделирует блок деклараций констант, позволяющий использовать генератор `iota`. Значение `iota` представляет собой номер строки в блоке, отсчитываемый от нуля. Пример из спецификации языка Go:

```
1 const (
2     a = 1 << iota // a == 1 (iota == 0)
3     b = 1 << iota // b == 2 (iota == 1)
4     c = 3          // c == 3 (iota == 2, unused)
5     d = 1 << iota // d == 8 (iota == 3)
6 )
```

2.5.4. Декларация функций и методов

```
1 (mot "function decl" :at "name" "function name"
2   :at "signature" "signature" :at "body" "block")
3 (mot "method decl" :at "receiver" "parameter decl"
4   :at "name" "method name" :at "signature" "signature"
5   :at "body" "block")
```

Метод отличается от функции наличием получателя (`"receiver"`), который связывает

метод с типом. Примеры:

```
1 func add(p *Point, q *Point) {  
2     p.x += q.x  
3 }  
4  
5 func (p *Point) add(q *Point) {  
6     p.x += q.x  
7 }
```

2.5.5. Декларация интерфейсов

Декларация интерфейса связывает имя с типом интерфейса:

```
1 (mot "interface decl" :at "name" "interface name" :at "type"  
    "interface type")
```

2.6. Выражения

Выражения в языке Go — это конструкции, которые после вычисления возвращают значение.

2.6.1. Общее понятие выражения

Общий тип выражения представлен как объединение конкретных типов выражений:

```
1 (typedef "expression" (uniont "literal" "conversion" "method  
    expr"  
2  
    "selector expr" "index expr"  
    "slice expr"  
3    "type assertion" "function call"  
4    "unary expression" "binary  
    expression"))
```

2.6.2. Приведение типов

Приведение типов (conversion) позволяет явно преобразовывать значения из одного типа в другой:

```

1 (mot "conversion" :at "type" "type" :at "expression"
   "expression")

```

Пример: `float64(2)` преобразует целое число в число с плавающей точкой.

2.6.3. Метод-выражения

Метод-выражения (method expression) создают функции на основе методов, преобразуя получатель в первый параметр функции:

```

1 (mot "method expression" :at "receiver type" "type" :at "name"
   "method name")

```

2.6.4. Выбор поля и метода

Выбор поля структуры или метода по экземпляру:

```

1 (typedef "selector expression" (uniont "selector struct"
   "selector method"))
2 (mot "selector struct" :at "struct" "expression" :at "field"
   "field name")
3 (mot "selector method" :at "method" "expression" :at "method"
   "method name")

```

Примеры: `person.Age` (выбор поля), `file.Close()` (вызов метода).

2.6.5. Индексирование и срезы

Индексирование применимо к массивам, срезам, картам и строкам:

```

1 (mot "index expr" :at "arr" "expression" :at "index"
   "expression")

```

Выражение среза (slice expression) создаёт новый срез из существующего массива, строки или среза:

```

1 (mot "slice expr" :at "arr" "expression" :at "low" "expression"
2   :at "high" "expression" :at "max" "expression")

```

Параметры `low`, `high` и `max` задают границы среза. Последний параметр может отсутствовать, в этом случае ёмкость среза определяется как `len(arr) - low`.

2.6.6. Проверка типа

Проверка типа (type assertion) применяется к интерфейсным значениям:

```
1 (mot "type assertion" :at "interface" "expression" :at "type"
   "type")
```

Пример:

```
1 var i interface{} = 3
2 num := i.(int) // num == 3, mun int
```

2.6.7. Вызов функции и операции с каналами

Вызов функции и операция чтения из канала:

```
1 (mot "function call" :at "function" "expression" :at
   "arguments" (listt "expression"))
2 (mot "<-1" :at 1 "expression")
```

2.6.8. Унарные выражения

Унарные алгебраические выражения описываются соответствующими типами:

```
1 (typedef "unary expression" (uniont "+1" "-1" "^1" "!1" "*1"
   "&1"))
2 (mot "+1" :at 1 "expression") ; +x == x
3 (mot "-1" :at 1 "expression") ; -x == -1 * x
4 (mot "^1" :at 1 "expression") ; поразрядное дополнение
5 (mot "!1" :at 1 "expression") ; логическое отрицание
6 (mot "*1" :at 1 "expression") ; разыменование указателя
7 (mot "&1" :at 1 "expression") ; взятие адреса
```

2.6.9. Бинарные выражения

Бинарные алгебраические выражения:

```

1  (typedef "binary expression" (uniont "1||2" "1&&2" "1*2" "1/2"
    "1%2" "1<<2" "1>>2"
2                                     "1&2" "1&^2" "1+2" "1-2"
    "1|2" "1^2"
3                                     "1==2" "1!=2" "1<2" "1<=2"
    "1>2" "1>=2"))
4  (mot "1||2" :at 1 "expression" :at 2 "expression")
5  (mot "1&&2" :at 1 "expression" :at 2 "expression")
6  (mot "1*2" :at 1 "expression" :at 2 "expression")
7  (mot "1/2" :at 1 "expression" :at 2 "expression")
8  (mot "1%2" :at 1 "expression" :at 2 "expression")
9  (mot "1<<2" :at 1 "expression" :at 2 "expression")
10 (mot "1>>2" :at 1 "expression" :at 2 "expression")
11 (mot "1&2" :at 1 "expression" :at 2 "expression")
12 (mot "1&^2" :at 1 "expression" :at 2 "expression")
13 (mot "1+2" :at 1 "expression" :at 2 "expression")
14 (mot "1-2" :at 1 "expression" :at 2 "expression")
15 (mot "1|2" :at 1 "expression" :at 2 "expression")
16 (mot "1^2" :at 1 "expression" :at 2 "expression")
17 (mot "1==2" :at 1 "expression" :at 2 "expression")
18 (mot "1!=2" :at 1 "expression" :at 2 "expression")
19 (mot "1<2" :at 1 "expression" :at 2 "expression")
20 (mot "1<=2" :at 1 "expression" :at 2 "expression")
21 (mot "1>2" :at 1 "expression" :at 2 "expression")
22 (mot "1>=2" :at 1 "expression" :at 2 "expression")

```

Семантика этих выражений соответствует стандартным определениям языка Go.

2.7. Операторы

Операторы (statements) в языке Go представляют собой конструкции, которые изменяют состояние программы, но не возвращают значений. В отличие от выражений, операторы выполняют действия: управляют потоком выполнения, осуществляют присваивания,

организуют циклы и условные переходы. Общее понятие оператора в модели представлено типом `"statement"`.

```

1 (typedef "statement" (uniont "declaration" "label stmt" "empty
    stmt" "1++" "1--"
2
3                                "assignment stmt" "if stmt"
                                "switch stmt" "for stmt"
4                                "return stmt" "break stmt"
                                "continue stmt" "goto stmt"
                                "go stmt" "1<-2" "fallthrough
                                stmt" "select stmt" "defer
                                stmt"))

```

2.7.1. Помеченные операторы

Помеченный оператор позволяет присвоить метку любому оператору программы. Метки используются для управления потоком выполнения с помощью операторов `break`, `continue` и `goto`.

```

1 (mot "label stmt" :at "label" "label name" :at "statement"
    "statement")

```

Пример использования метки в языке Go:

```

1 OuterLoop:
2     for i := 0; i < 10; i++ {
3         for j := 0; j < 10; j++ {
4             if i*j > 50 {
5                 break OuterLoop
6             }
7         }
8     }

```

В данном примере метка `OuterLoop` позволяет оператору `break` выйти не только из внутреннего, но и из внешнего цикла.

2.7.2. Пустой оператор

Пустой оператор не выполняет никаких действий. Он используется в случаях, когда синтаксис требует наличия оператора, но никаких действий выполнять не требуется.

```
1 (typedef "empty stmt")
```

2.7.3. Операторы инкремента и декремента

В языке Go операторы инкремента и декремента являются операторами, а не выражениями, что отличает Go от многих других языков программирования. Они могут использоваться только в постфиксной форме и не возвращают значения.

```
1 (mot "1++ stmt" :at "1" "expression") ; x += 1
2 (mot "1-- stmt" :at "1" "expression") ; x -= 1
```

Примеры использования:

```
1 x++
2 y--
```

2.7.4. Операторы присваивания

Операторы присваивания включают простое присваивание и составные присваивания с арифметическими и битовыми операциями. В языке Go допускается множественное присваивание, когда левая и правая части содержат одинаковое количество выражений.

```
1 (typedef "assignment stmt" (uniont "1=2" "1+=2" "1-=2" "1|=2"
    "1^=2"
    "1*=2" "1/=2" "1%=2"
    "1<=2" "1>=2"
    "1&=2" "1&^=2"))
2
3
4 (mot "1=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
5 (mot "1+=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
6 (mot "1-=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
7 (mot "1|=2" :at 1 (listt "expression") :at 2 (listt
```

```

    "expression"))
8 (mot "1^=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
9 (mot "1*=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
10 (mot "1/=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
11 (mot "1%=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
12 (mot "1<=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
13 (mot "1>=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
14 (mot "1&=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))
15 (mot "1^=2" :at 1 (listt "expression") :at 2 (listt
    "expression"))

```

Примеры присваиваний:

```

1 x, y = y, x      // множественное присваивание
2 a += 10          // составное присваивание со сложением
3 b &= 0xFF        // составное присваивание с побитовым И

```

2.7.5. Условный оператор if

Условный оператор **if** может содержать необязательную инициализирующую часть (простой оператор), условие и ветви **then** и **else**. Инициализирующая часть выполняется до проверки условия и позволяет объявлять переменные, область видимости которых ограничена конструкцией **if**.

```

1 (mot "if stmt" :at "init" "statement" :at "condition"
    "expression"
2   :at "then" "block" :at "else" (uniont "if statement"

```



```
"block"))
```

Пример использования оператора `if` с инициализацией:

```
1 if x := f(); x < y {
2     return x, true
3 } else if x > y {
4     return y, true
5 }
```

В данном примере переменная `x` доступна только внутри конструкции `if` и её ветвей.

2.7.6. Оператор выбора switch

Оператор `switch` в языке Go существует в двух формах: переключатель по выражению и переключатель по типу. В отличие от многих других языков, ветви `switch` в Go не проваливаются по умолчанию в следующую ветвь.

```
1 (typedef "switch stmt" (uniont "expr switch stmt" "type switch
   stmt"))
2 (mot "expr switch stmt" :at "init" "statement" :at
   "controlling expression" "expression"
3   :at "cases" (listt "expr case clause"))
4 (mot "expr case clause" :at "cases" (uniont (listt
   "expression") "default")
5   :at "statements" (listt "statement"))
6 (cot "default")
```

Переключатель по выражению позволяет выбрать ветвь на основе значения управляющего выражения:

```
1 switch a {
2 case 0, 1, 2:
3     f()
4 default:
5     g()
6 }
```

Переключатель по типу используется для проверки динамического типа значения интерфейса:

```

1 (mot "type switch stmt" :at "init" "statement" :at "guard"
   "type switch guard"
2   :at "cases" (listt "type case clause"))
3 (mot "type switch guard" :at "name" "type name" :at "variable"
   "expression")
4 (mot "type case clause" :at "cases" (uniont (listt "type")
   "default")
5   :at "statements" (listt "statement"))

```

Пример переключателя по типу:

```

1 switch t := x.(type) {
2 case nil:
3     fmt.Println("x is nil")
4 default:
5     fmt.Println("don't know the type")
6 }

```

2.7.7. Оператор цикла for

В языке Go существует единственная циклическая конструкция — оператор `for`, который может быть представлен в двух формах: классический цикл с условием и цикл с оператором `range`.

```

1 (typedef "for statement" (uniont "for condition" "for range"))
2 (mot "for condition" :at "init" "statement" :at "condition"
   "expression"
3   :at "post" "statement" :at "body" "block")
4 (mot "for range" :at "names" (listt "variable name")
5   :at "arr" "expression" :at "body" "block")

```

Классический цикл `for` может использоваться как цикл со счётчиком, как цикл `while` (если опущены инициализация и пост-действие), а также как бесконечный цикл (если опущено условие):

Цикл с оператором `range` предназначен для итерации по элементам составных типов данных: массивов, срезов, строк, карт и каналов.

2.7.8. Операторы передачи управления

К операторам передачи управления относятся `return`, `break`, `continue` и `goto`. Эти операторы изменяют естественный поток выполнения программы.

```
1 (mot "return" :at "expressions" (listt "expression"))
2 (mot "break" :at "label" "label")
3 (mot "continue" :at "label" "label")
4 (mot "goto" :at "label" "label")
```

Оператор `return` завершает выполнение функции и возвращает указанные значения. В функциях, не возвращающих значений, оператор `return` может использоваться без аргументов:

```
1 func add(a, b int) int {
2     return a + b
3 }
```

Операторы `break` и `continue` могут использоваться с метками для выхода из вложенных циклов:

```
1 OuterLoop:
2     for {
3         for {
4             break OuterLoop
5         }
6     }
```

Оператор `goto` осуществляет безусловный переход к указанной метке. Его использование в Go ограничено: переход не может перепрыгивать через объявления переменных.

2.7.9. Оператор go

Оператор `go` запускает функцию в отдельной горутине — независимом потоке управления, который выполняется конкурентно с другими горутинами.

```
1 (mot "go stmt" :at "body" "expression")
```

Пример запуска горутин:

```
1 go Server()
2 go func(ch chan<- bool) {
3     for {
4         sleep(10)
5         ch <- true
6     }
7 }(c)
```

2.7.10. Оператор отправки в канал

Оператор `<-` используется для отправки данных в канал или чтения данных из канала. В модели оператор отправки представлен отдельным типом.

```
1 (mot "1<-2" :at 1 "expression")
```

Пример отправки значения в канал:

```
1 ch <- 42
```

2.7.11. Оператор select

Оператор `select` позволяет горутине ожидать выполнения нескольких операций с каналами одновременно. Выполняется та ветвь, операция в которой готова к выполнению первой. Если готово несколько ветвей, выбор происходит псевдослучайным образом.

```
1 (mot "select stmt" :at "statement" (listt "common clause"))
2 (mot "common clause" :at "case" (uniont "send stmt" "receive =
    stmt"
3                                     "receive := stmt" "default"))
4     :at "statements" (listt "statement"))
5 (mot "receive = stmt" :at "names" (listt "expression") :at
```

```

    "expression" "expression")
6 (mot "receive := stmt" :at "names" (listt "variable name") :at
    "expression" "expression")

```

Пример использования `select`:

```

1 ch1, ch2 := make(chan string), make(chan string)
2 go func() { time.Sleep(1 * time.Second); ch1 <- "данные из кана
    ла 1" }()
3 go func() { time.Sleep(2 * time.Second); ch2 <- "данные из кана
    ла 2" }()
4
5 select {
6 case msg1 := <-ch1:
7     fmt.Println("Получено:", msg1)
8 case msg2 := <-ch2:
9     fmt.Println("Получено:", msg2)
10 }

```

2.7.12. Оператор fallthrough

Оператор `fallthrough` передаёт управление следующей ветви в операторе `switch`. В отличие от многих языков, где проваливание происходит по умолчанию, в Go оно должно быть явно указано с помощью `fallthrough`.

```

1 (typedef "fallthrough" (enumt "fallthrough"))

```

Пример использования:

```

1 switch {
2 case true:
3     fmt.Print("true & ")
4     fallthrough
5 case false:
6     fmt.Print("false")
7 }

```

2.7.13. Оператор defer

Оператор `defer` откладывает выполнение функции до момента возврата из окружающей функции. Отложенные функции выполняются в порядке LIFO (последним пришёл — первым ушёл), что часто используется для освобождения ресурсов.

```
1 (mot "defer stmt" :at "expression" "expression")
```

Пример использования `defer`:

```
1 for i := 0; i <= 3; i++ {  
2     defer fmt.Print(i) // prints 3 2 1 0  
3 }
```

3. Заключение

В работе представлена онтологическая модель фрагмента языка Go, реализованная на языке ABML. Предложенный подход позволяет формально описывать семантику языковых конструкций и может быть использован для разработки инструментов верификации.

4. Список литературы